# ESTRUTURA DE DADOS

## Pilhas e Filas

Instituto Federal do Triângulo Mineiro – IFTM

| | |
|---|---|
| **Curso:** | Tecnologia em Sistemas para Internet |
| **Disciplina:** | Estrutura de Dados |
| **Semestre:** | 2026/1 |
| **Pré-requisitos:** | OO/Generics, Complexidade, Recursão, TADs, Arrays Dinâmicos, Listas Encadeadas |

## 1. Objetivos de Aprendizagem

### 1.1 Objetivo Geral

Compreender Pilhas e Filas como Tipos Abstratos de Dados com restrições de acesso que definem comportamentos específicos — LIFO e FIFO, respectivamente. Implementar as duas estruturas em Java com arrays e com listas encadeadas, compará-las com as interfaces Deque<T> e Queue<T> do Java Collections Framework e identificar os padrões de aplicação mais comuns no desenvolvimento de sistemas web com Spring Boot.

### 1.2 Objetivos Específicos

- Diferenciar o comportamento LIFO (Last In, First Out) da Pilha do comportamento FIFO (First In, First Out) da Fila, identificando as operações que cada TAD expõe.
- Implementar PilhaArray<T> com array redimensionável e PilhaLista<T> com lista simplesmente encadeada, reutilizando Node<T> da Semana 5.
- Implementar FilaCircular<T> com array circular (ponteiros inicio/fim e operador módulo) e FilaLista<T> com lista duplamente encadeada.
- Analisar a complexidade das operações push, pop, enqueue, dequeue e peek usando notação Big-O e Θ, distinguindo custo real de custo amortizado.
- Usar as interfaces Deque<T> e Queue<T> do JCF, escolhendo entre ArrayDeque, LinkedList e PriorityQueue com justificativa técnica.
- Aplicar Pilha e Fila em problemas reais de sistemas web: verificação de balanceamento, avaliação de expressões, rate limiting, histórico de navegação e processamento assíncrono.
- Escrever suítes de testes com JUnit 5 cobrindo casos normais, casos de borda e cenários de erro para cada estrutura implementada.

## 2. Contexto Profissional

Pilhas e Filas são as estruturas de dados mais usadas no cotidiano de quem desenvolve sistemas web — mesmo quando não são reconhecidas pelo nome. Elas aparecem de forma explícita no código e de forma implícita na arquitetura de frameworks e servidores. Os quatro cenários a seguir ilustram esse uso com referências ao ecossistema Java/Spring Boot.

### Cenário 1 — Desfazer/Refazer em editors online (Pilha)

Qualquer editor de texto colaborativo — Google Docs, Notion, VS Code com Live Share — mantém duas pilhas para as operações **Ctrl+Z** (undo) e **Ctrl+Y** (redo). Cada edição é empilhada na pilha de histórico. Ao desfazer, o item sai da pilha de histórico e é empilhado na pilha de redo. Ao editar texto novo após desfazer, a pilha de redo é zerada. Em uma API REST, isso equivale a manter uma **Deque<Command>** por sessão, aplicando o Padrão de Projeto Command com Pilha.

### Cenário 2 — Processamento assíncrono de pedidos (Fila)

Em um e-commerce com Spring Boot, o usuário finaliza a compra e recebe resposta HTTP 202 Accepted imediatamente. O processamento real — cálculo de frete, reserva de estoque, envio de e-mail de confirmação — acontece de forma assíncrona. O pedido é colocado em uma fila e workers o processam na ordem de chegada (FIFO). O RabbitMQ com Spring AMQP é, em essência, uma fila distribuída e persistente. Entender a semântica da Fila é o pré-requisito para usar essas ferramentas com propriedade.

### Cenário 3 — Pilha de chamadas da JVM e diagnóstico de produção (Pilha)

A própria JVM mantém uma pilha de execução (call stack) para gerenciar a ativação de métodos. Ao analisar uma stack trace de **NullPointerException** em um log de produção, você está lendo o conteúdo dessa pilha do topo (método onde o erro ocorreu) até a base (método main ou o thread de entrada). Ferramentas como Elastic APM e Sentry processam essas stack traces usando lógica baseada em pilha para agrupar erros e identificar a raiz do problema.

### Cenário 4 — Fila de requisições HTTP e configuração do Tomcat (Fila)

O Tomcat, servidor de aplicação embarcado padrão do Spring Boot, mantém um pool de threads e uma fila de requisições HTTP. Quando todos os threads estão ocupados, novas requisições aguardam na fila. O parâmetro **server.tomcat.accept-count** no **application.properties** configura o tamanho máximo dessa fila. Uma fila transbordando gera HTTP 503 (Service Unavailable). Compreender a estrutura por trás desse parâmetro é o que permite dimensionar corretamente um serviço antes de ir para produção.

## 3. Fundamentos Teóricos

### 3.1 O TAD Pilha — comportamento LIFO

Uma Pilha é um TAD que organiza elementos em sequência linear e restringe o acesso ao elemento mais recentemente inserido. O princípio LIFO (Last In, First Out) significa que o último elemento a entrar é sempre o primeiro a sair. Analogias físicas são imediatas: uma pilha de pratos, uma pilha de documentos sobre a mesa.

**Operações do contrato (interface Pilha<T>):**

- **push(T elemento)** — insere no topo. Custo: $\Theta(1)$ amortizado.

- **pop()** — remove e retorna o elemento do topo. Custo: Θ(1).
- **peek()** — consulta o topo sem remover. Custo: Θ(1).
- **isEmpty()** — verifica se não há elementos. Custo: Θ(1).
- **size()** — retorna a contagem de elementos. Custo: Θ(1).

> **Invariante da Pilha**
>
> O único elemento acessível em qualquer momento é o topo. Não há acesso por índice nem operação de busca. Essa restrição é intencional: ao eliminar essas operações do contrato, o TAD Pilha garante que qualquer implementação (com array, com lista encadeada, com arquivo em disco) pode oferecer as operações essenciais em tempo constante, e que o código-cliente é forçado a pensar no padrão LIFO — o que torna a lógica mais previsível e fácil de depurar.

## 3.2 O TAD Fila — comportamento FIFO

Uma Fila é um TAD que organiza elementos em sequência linear e restringe as inserções ao final e as remoções ao início. O princípio FIFO (First In, First Out) garante que o elemento que aguarda há mais tempo é sempre o próximo a ser atendido. Analogias: fila de banco, fila de impressão, fila de embarque em aeroporto.

**Operações do contrato (interface Fila<T>):**

- **enqueue(T elemento)** — insere no final. Custo: Θ(1).
- **dequeue()** — remove e retorna o elemento do início. Custo: Θ(1).
- **peek()** — consulta o início sem remover. Custo: Θ(1).
- **isEmpty()** e **size()** — Θ(1).

## 3.3 Relação com o TAD Lista

Tanto Pilha quanto Fila são especializações do TAD Lista que impõem restrições de acesso. O TAD Lista, definido na Semana 4, permite inserção e remoção em qualquer posição. Ao remover essas operações do contrato e expor apenas as extremidades, obtemos estruturas com semântica mais restrita e mais expressiva — o programador que usa uma Pilha comunica explicitamente no código que está trabalhando com ordem reversa; quem usa uma Fila comunica que respeita a ordem de chegada. Essa expressividade reduz bugs, porque o tipo impede usos acidentais fora do padrão.

## 3.4 Deque — a superestrutura

Um Deque (Double-Ended Queue, pronúncia "deck") permite inserção e remoção em **ambas as extremidades** em Θ(1). Ele é uma superestrutura que contém Pilha e Fila como casos especiais: restrinja o acesso a uma extremidade e você tem uma Pilha; use uma extremidade para inserir e a outra para remover e você tem uma Fila. O JCF modelou essa relação colocando a interface **Deque<T>** no centro de toda a hierarquia de Pilhas e Filas — substituindo a classe legada **Stack** e sendo a implementação recomendada para ambos os TADs.

# 4. Implementação em Java

## 4.1 Definindo os contratos — interfaces Pilha<T> e Fila<T>

Definimos primeiro o contrato (TAD) antes de qualquer implementação concreta. Isso garante que o código-cliente seja acoplado à interface, não à implementação — princípio central de bom design em Java.

```
package estruturas;

import java.util.EmptyStackException;

/**
 * TAD Pilha — coleção LIFO (Last In, First Out).
 * Toda implementação deve garantir push/pop/peek em O(1).
 *
 * @param <T> tipo dos elementos
 */
public interface Pilha<T> {

    /** Empilha elemento no topo. O(1) amortizado. */
    void push(T elemento);

    /** Remove e retorna o topo. O(1). Lança EmptyStackException se vazia. */
    T pop();

    /** Retorna o topo sem remover. O(1). */
    T peek();

    /** true se não houver elementos. */
    boolean isEmpty();

    /** Número de elementos. */
    int size();
}
```

```
package estruturas;

import java.util.NoSuchElementException;

/**
 * TAD Fila — coleção FIFO (First In, First Out).
 * Toda implementação deve garantir enqueue/dequeue/peek em O(1).
 *
 * @param <T> tipo dos elementos
 */
```

```
public interface Fila<T> {

    /** Insere elemento no final da fila. O(1). */
    void enqueue(T elemento);

    /** Remove e retorna o elemento do início. O(1). */
    T dequeue();

    /** Retorna o início sem remover. O(1). */
    T peek();

    boolean isEmpty();
    int size();
}
```

## 4.2 Pilha com Array Dinâmico — PilhaArray<T>

A implementação mais direta usa um array interno. O topo da pilha corresponde ao último índice ocupado: o campo topo armazena o índice do próximo slot livre (não o do último elemento inserido). Inserção e remoção sempre ocorrem nessa extremidade, o que elimina qualquer deslocamento de elementos e garante Θ(1) para ambas as operações. O array dobra de tamanho quando esgota a capacidade.

```
package estruturas;

import java.util.Arrays;
import java.util.EmptyStackException;

public class PilhaArray<T> implements Pilha<T> {

    private static final int CAP_INICIAL = 16;
    private Object[] elementos;   // array interno — não pode ser T[] por type
erasure
    private int topo;             // índice do próximo slot LIVRE (não do último
elemento)

    public PilhaArray() {
        elementos = new Object[CAP_INICIAL];
        topo = 0;
    }

    // ── push: insere no índice topo e incrementa o ponteiro ──────────────
    @Override   // Θ(1) amortizado
    public void push(T elemento) {
        garantirCapacidade();
        elementos[topo] = elemento;
```

```java
        topo++;
    }

    // ── pop: decrementa topo, retorna e anula a referência ──────────────
    @Override   // Θ(1)
    @SuppressWarnings("unchecked")
    public T pop() {
        if (isEmpty()) throw new EmptyStackException();
        topo--;
        T removido = (T) elementos[topo];
        elementos[topo] = null;   // libera referência — evita memory leak
        return removido;
    }

    // ── peek: lê sem alterar o ponteiro ──────────────────────────────────
    @Override   // Θ(1)
    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) throw new EmptyStackException();
        return (T) elementos[topo - 1];
    }

    @Override public boolean isEmpty() { return topo == 0; }
    @Override public int size()        { return topo; }

    // ── redimensionamento: dobra o array quando esgota a capacidade ──
    private void garantirCapacidade() {
        if (topo == elementos.length) {
            // Arrays.copyOf aloca novo array e copia — O(n), mas amortizado
            elementos = Arrays.copyOf(elementos, elementos.length * 2);
        }
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("Pilha[topo→");
        for (int i = topo - 1; i >= 0; i--) {
            if (i < topo - 1) sb.append(", ");
            sb.append(elementos[i]);
        }
        return sb.append("]").toString();
    }
}
```

**Por que Object[] e não T[]?**

Java apaga informações de tipo genérico em tempo de execução (*type erasure*): T se torna Object no bytecode. Por isso, **new T[10]** não compila — o compilador não sabe qual tipo

alocar. A solução adotada pelo próprio **ArrayList** do JCF é usar **Object[]** internamente e fazer cast nas operações de leitura, suprimindo o aviso do compilador com **@SuppressWarnings("unchecked")**. O cast é seguro porque a única forma de inserir elementos é via **push(T)**, que já garante o tipo correto.

## 4.3 Pilha com Lista Encadeada — PilhaLista<T>

A alternativa com lista encadeada elimina completamente o redimensionamento. A cabeça da lista atua como topo da pilha: **push** insere um nó na cabeça (O(1) puro, sem custo ocasional) e **pop** remove a cabeça (O(1)). O custo extra é a alocação de um novo **Node<T>** na heap a cada push — um trade-off que favorece previsibilidade de latência em detrimento de uso de memória levemente maior.

```
package estruturas;

import java.util.EmptyStackException;

public class PilhaLista<T> implements Pilha<T> {

    private Node<T> topo;      // referência para o nó no topo
    private int quantidade;

    public PilhaLista() {
        topo = null;
        quantidade = 0;
    }

    // ── push: cria nó cujo .proximo aponta para o topo atual ──────────
    @Override   // Θ(1) — sem amortização, sem redimensionamento
    public void push(T elemento) {
        // new Node<>(dado, proximoNó)
        topo = new Node<>(elemento, topo);
        quantidade++;
    }

    // ── pop: salva dado, avança topo para o próximo nó ────────────────
    @Override   // Θ(1)
    public T pop() {
        if (isEmpty()) throw new EmptyStackException();
        T removido = topo.dado;
        topo = topo.proximo;   // GC coleta o nó desconectado
        quantidade--;
        return removido;
    }

    @Override   // Θ(1)
    public T peek() {
```

```
        if (isEmpty()) throw new EmptyStackException();
        return topo.dado;
    }

    @Override public boolean isEmpty() { return topo == null; }
    @Override public int size()        { return quantidade; }
}
```

## 4.4 Fila com Array Circular — FilaCircular<T>

A implementação de fila com array tem uma armadilha clássica: se tratarmos o array de forma linear, cada dequeue() precisaria deslocar todos os elementos uma posição para a esquerda, custando O(n). A solução é o array circular: dois ponteiros inteiros — inicio (onde sai) e fim (onde entra) — circulam pelo array usando o operador módulo. Quando um ponteiro chega ao final do array, ele retorna automaticamente ao índice 0, sem mover nenhum elemento.

**Como funciona o array circular**

Imagine o array como uma esteira rolante circular. O ponteiro inicio aponta para o item que saiu primeiro; fim aponta para o slot vazio onde o próximo item entra. A fórmula fim = (fim + 1) % capacidade garante que ao ultrapassar o último índice, fim volta ao índice 0 — sem cópia e sem deslocamento. A fila está cheia quando quantidade == elementos.length.

```
package estruturas;

import java.util.Arrays;
import java.util.NoSuchElementException;

public class FilaCircular<T> implements Fila<T> {

    private static final int CAP_INICIAL = 16;
    private Object[] elementos;
    private int inicio;     // índice do próximo elemento a sair
    private int fim;        // índice do próximo slot livre para entrar
    private int quantidade;

    public FilaCircular() {
        elementos = new Object[CAP_INICIAL];
        inicio = 0;
        fim    = 0;
        quantidade = 0;
    }

    // ── enqueue: insere em fim, avança fim circularmente ──────────────
    @Override   // Θ(1) amortizado
    public void enqueue(T elemento) {
```

```
        if (quantidade == elementos.length) redimensionar();
        elementos[fim] = elemento;
        fim = (fim + 1) % elementos.length;  // wrap-around
        quantidade++;
    }

    // ── dequeue: retorna inicio, avança inicio circularmente ──────────────
    @Override   // Θ(1)
    @SuppressWarnings("unchecked")
    public T dequeue() {
        if (isEmpty()) throw new NoSuchElementException("Fila vazia");
        T removido = (T) elementos[inicio];
        elementos[inicio] = null;                // libera referência
        inicio = (inicio + 1) % elementos.length;
        quantidade--;
        return removido;
    }

    @Override   // Θ(1)
    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) throw new NoSuchElementException("Fila vazia");
        return (T) elementos[inicio];
    }

    @Override public boolean isEmpty() { return quantidade == 0; }
    @Override public int size()        { return quantidade; }

    /**
     * Redimensiona para o dobro. Realinha os elementos a partir do índice 0.
     * O realinhamento é necessário porque os elementos podem estar fragmentados
     * em duas partes do array circular (ex.: [40,50,_,_,10,20,30] com
inicio=4).
     */
    private void redimensionar() {
        Object[] novo = new Object[elementos.length * 2];
        for (int i = 0; i < quantidade; i++) {
            // (inicio + i) % length mapeia a posição lógica para a física
            novo[i] = elementos[(inicio + i) % elementos.length];
        }
        inicio    = 0;
        fim       = quantidade;
        elementos = novo;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("Fila[frente→");
```

```
        for (int i = 0; i < quantidade; i++) {
            if (i > 0) sb.append(", ");
            sb.append(elementos[(inicio + i) % elementos.length]);
        }
        return sb.append("]").toString();
    }
}
```

## 4.5 Fila com Lista Encadeada — FilaLista<T>

Com a lista encadeada, a implementação é direta: **enqueue** insere no final usando a referência cauda (Θ(1)) e **dequeue** remove do início usando a referência cabeca (Θ(1)).

```
package estruturas;

import java.util.NoSuchElementException;

public class FilaLista<T> implements Fila<T> {

    private Node<T> cabeca;   // primeiro a sair (frente da fila)
    private Node<T> cauda;    // último a entrar (fundo da fila)
    private int quantidade;

    public FilaLista() { cabeca = null; cauda = null; quantidade = 0; }

    // ── enqueue: novo nó vai para após a cauda atual ──────────────────
    @Override   // Θ(1)
    public void enqueue(T elemento) {
        Node<T> novo = new Node<>(elemento);
        if (isEmpty()) {
            cabeca = novo;
        } else {
            cauda.proximo = novo;   // encadeia o novo nó ao final
        }
        cauda = novo;
        quantidade++;
    }

    // ── dequeue: remove cabeca e avança o ponteiro ────────────────────
    @Override   // Θ(1)
    public T dequeue() {
        if (isEmpty()) throw new NoSuchElementException("Fila vazia");
        T removido = cabeca.dado;
        cabeca = cabeca.proximo;
        if (cabeca == null) cauda = null;   // lista ficou vazia
        quantidade--;
```

```
        return removido;
    }

    @Override public T peek() {
        if (isEmpty()) throw new NoSuchElementException("Fila vazia");
        return cabeca.dado;
    }

    @Override public boolean isEmpty() { return quantidade == 0; }
    @Override public int size()        { return quantidade; }
}
```

## 4.6 Aplicações Práticas com Código

### 4.6.1 Verificador de Balanceamento de Expressões

O problema de verificar se parênteses, colchetes e chaves estão corretamente balanceados é um dos usos mais clássicos da Pilha. Ele aparece em compiladores, parsers de JSON/XML, e em IDEs como o VS Code — que usa esse algoritmo para sublinhar erros de sintaxe em tempo real.

```
/**
 * Verifica balanceamento de delimitadores em uma expressão Java/JSON.
 * Algoritmo: O(n) temporal, O(n) espacial (pilha no pior caso).
 */
public class VerificadorBalanceamento {

    public static boolean estaBalanceado(String expressao) {
        Pilha<Character> pilha = new PilhaArray<>();

        for (char c : expressao.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') {
                pilha.push(c);                  // abre: empilha
            } else if (c == ')' || c == ']' || c == '}') {
                // fecha: pilha vazia → faltou abrir
                if (pilha.isEmpty()) return false;
                if (!corresponde(pilha.pop(), c)) return false;
            }
            // outros caracteres (letras, operadores) são ignorados
        }
        // se a pilha não ficou vazia, há delimitadores abertos sem fechar
        return pilha.isEmpty();
    }

    private static boolean corresponde(char aberto, char fechado) {
        return (aberto == '(' && fechado == ')')
            || (aberto == '[' && fechado == ']')
            || (aberto == '{' && fechado == '}');
```

```
    }

    public static void main(String[] args) {
        System.out.println(estaBalanceado("{\"key\": [1, 2, 3]}"));  // true
        System.out.println(estaBalanceado("{[}]"));                  // false
        System.out.println(estaBalanceado("((("));                   // false
    }
}
```

### 4.6.2 Simulação de Fila de Pedidos de E-commerce

O exemplo a seguir simula o padrão produtor-consumidor com FilaLista<T>. O produtor (thread de recebimento HTTP) adiciona pedidos; o consumidor (worker assíncrono) os processa na ordem FIFO. Esse é o modelo conceitual exato do Spring AMQP com RabbitMQ.

```
// Record Java 16+: classe imutável de dados em uma linha
record Pedido(int id, String produto, double valor) {}

public class SimulacaoPedidos {
    public static void main(String[] args) {
        Fila<Pedido> fila = new FilaLista<>();

        // Produtor: recebe pedidos via HTTP
        fila.enqueue(new Pedido(1, "Notebook",  3500.00));
        fila.enqueue(new Pedido(2, "Mouse",      150.00));
        fila.enqueue(new Pedido(3, "Teclado",    280.00));

        System.out.println("Pedidos na fila: " + fila.size());
        System.out.println("Próximo: " + fila.peek().produto());

        // Consumidor: processa em ordem FIFO
        while (!fila.isEmpty()) {
            Pedido p = fila.dequeue();
            System.out.printf("Processado #%d: %s — R$ %.2f%n",
                p.id(), p.produto(), p.valor());
        }
    }
}
// Saída:
// Pedidos na fila: 3
// Próximo: Notebook
// Processado #1: Notebook — R$ 3500,00
// Processado #2: Mouse — R$ 150,00
// Processado #3: Teclado — R$ 280,00
```

### 4.6.3 Histórico de Navegação com Pilha Dupla

Browsers implementam Voltar e Avançar com duas pilhas. Toda página visitada vai para a pilha de histórico. Ao clicar Voltar, a página atual vai para a pilha de futuro. Ao visitar nova página após voltar, a pilha de futuro é limpa — exatamente como em qualquer browser real.

```
public class HistoricoNavegacao {

    // Usamos Deque<String> do JCF — demonstra integração com a seção 6
    private final java.util.Deque<String> historico = new java.util.ArrayDeque<>();
    private final java.util.Deque<String> futuro    = new java.util.ArrayDeque<>();
    private String paginaAtual;

    public void visitar(String url) {
        if (paginaAtual != null) historico.push(paginaAtual);
        paginaAtual = url;
        futuro.clear();   // nova navegação descarta o histórico de "avançar"
    }

    public String voltar() {
        if (historico.isEmpty())
            throw new IllegalStateException("Sem histórico anterior");
        futuro.push(paginaAtual);
        return paginaAtual = historico.pop();
    }

    public String avancar() {
        if (futuro.isEmpty())
            throw new IllegalStateException("Sem páginas futuras");
        historico.push(paginaAtual);
        return paginaAtual = futuro.pop();
    }

    public String paginaAtual()    { return paginaAtual; }
    public boolean podeVoltar()    { return !historico.isEmpty(); }
    public boolean podeAvancar()   { return !futuro.isEmpty(); }
}
```

## 5. Análise de Complexidade

Toda operação de Pilha e Fila deve ser O(1). A diferença entre as implementações está no tipo de custo — constante puro versus amortizado — e no overhead de memória por elemento.

| Operação | PilhaArray | PilhaLista | FilaCircular | FilaLista |
|---|---|---|---|---|
| push / enqueue | Θ(1)* | Θ(1) | Θ(1)* | Θ(1) |
| pop / dequeue | Θ(1) | Θ(1) | Θ(1) | Θ(1) |

| Operação | PilhaArray | PilhaLista | FilaCircular | FilaLista |
|---|---|---|---|---|
| peek | **Θ(1)** | **Θ(1)** | **Θ(1)** | **Θ(1)** |
| Redimensionamento (custo pontual) | **O(n)** | *não aplica* | **O(n)** | *não aplica* |
| Overhead de memória/elemento | **1 ref.** | **2 refs.** | **1 ref.** | **2 refs.** |

** Θ(1) amortizado — ocasionalmente Θ(n) quando ocorre redimensionamento do array interno. Verde: melhor; vermelho: pior ou custo adicional.*

## 5.1 Complexidade Amortizada em Detalhe

O conceito de *complexidade amortizada* aparece quando uma operação é ocasionalmente cara, mas o custo médio sobre uma sequência longa de operações é baixo. Para **PilhaArray** e **FilaCircular**, o redimensionamento custa Θ(n) no pior caso pontual, mas ocorre cada vez menos frequentemente: após n, 2n, 4n, 8n inserções. O custo total de n inserções é Θ(n), o que dá Θ(1) por inserção na média.

Para visualizar: inserindo 8 elementos em uma PilhaArray que começa com capacidade 2:

| Inserção | Capacidade antes | Redimensiona? | Cópias realizadas | Custo total (ops) |
|---|---|---|---|---|
| 1ª | 2 | Não | 0 | 1 |
| 2ª | 2 | Não | 0 | 1 |
| 3ª | 2 → 4 | Sim | 2 | 3 |
| 4ª | 4 | Não | 0 | 1 |
| 5ª | 4 → 8 | Sim | 4 | 5 |
| 6ª | 8 | Não | 0 | 1 |
| 7ª | 8 | Não | 0 | 1 |
| 8ª | 8 | Não | 0 | 1 |
| Total | — | — | 6 cópias | 14 operações |

Custo médio: 14 operações / 8 inserções = 1,75 operações por inserção. Isso é Θ(1) amortizado. A chave está no fator de crescimento: dobrar a capacidade garante que o custo das cópias se dilui geometricamente sobre as inserções subsequentes.

## 5.2 Array vs. Lista Encadeada — quando cada um é melhor

A análise assintótica não conta toda a história. Na prática, o hardware influencia o desempenho observado de formas que Big-O não captura.

**Localidade de cache (cache locality)**

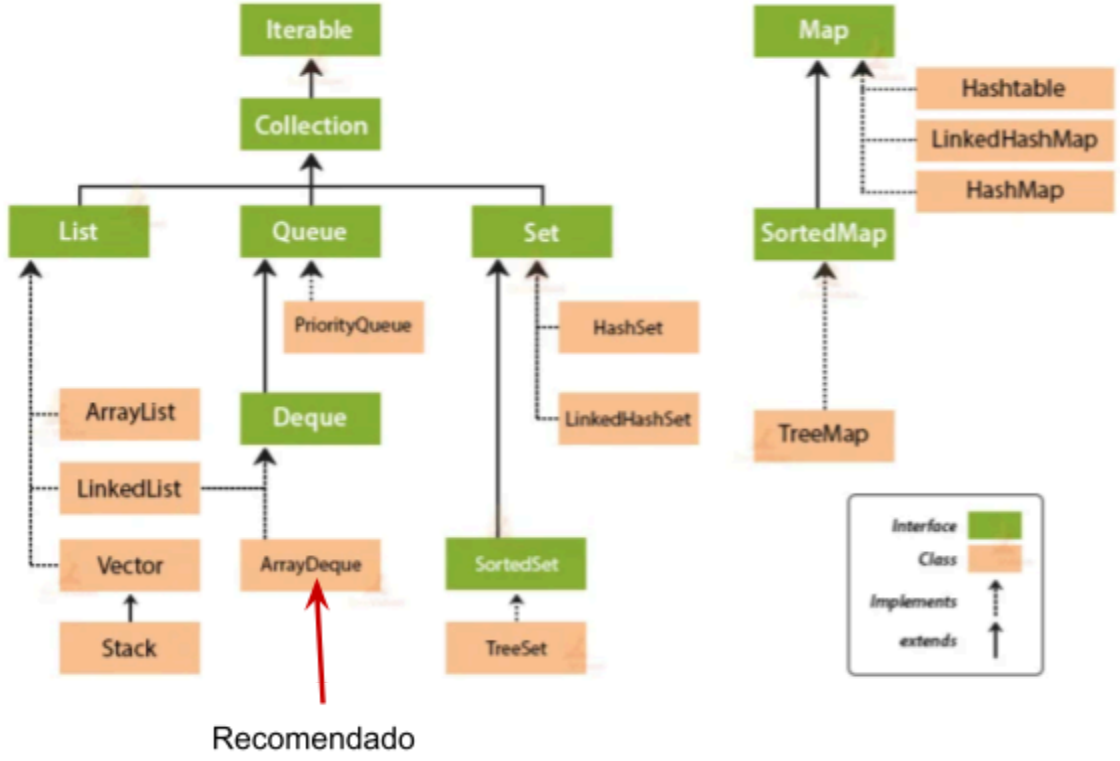
Arrays armazenam elementos em posições contíguas de memória. Ao acessar um elemento, o processador carrega uma linha de cache com vários elementos vizinhos. Na operação seguinte, esses dados já estão no cache — o que é muito mais rápido do que buscar na memória principal. Em listas encadeadas, cada Node<T> está em uma posição aleatória da heap; o processador precisa buscar cada nó separadamente. Para operações em sequência, isso pode fazer **PilhaArray** ser 2–5× mais rápida que **PilhaLista** mesmo ambas tendo Θ(1) assintótico.

**Use implementação com Array quando:** o número máximo de elementos é previsível, a latência uniforme é dispensável (tolera o pico ocasional de redimensionamento) e a aplicação faz sequências longas de push/pop — o cache locality compensa.

**Use implementação com Lista quando:** o volume de elementos é imprevisível e você precisa garantir que nenhuma operação individual tenha pico de latência (ex.: sistemas de tempo real ou de baixa latência), ou quando o Node<T> já está disponível e o custo de alocação de objetos é aceitável.

## 6. Comparação com o Java Collections Framework

### 6.1 A Hierarquia de Interfaces — Queue<T> e Deque<T>




O JCF não disponibiliza uma interface separada chamada Stack. Em vez disso, toda a hierarquia de Pilhas e Filas está centralizada em **Deque<T>** (Double-Ended Queue), que por sua vez estende **Queue<T>**. A interface **Queue<T>** define o comportamento FIFO; **Deque<T>** acrescenta operações nas duas extremidades, tornando-se uma superestrutura que engloba os dois TADs.

## 6.2 API de Queue<T> — pares de métodos

Cada operação da **Queue<T>** existe em dois sabores: um que lança exceção em caso de falha (fila vazia / cheia) e outro que retorna **null** ou **false**. A tabela abaixo compara os pares com os métodos das nossas interfaces.

| Operação | Lança exceção | Retorna null/false | Nosso método |
|---|---|---|---|
| Inserir no final | add(e) | offer(e) | enqueue(e) |
| Remover do início | remove() | poll() | dequeue() |
| Consultar início | element() | peek() | peek() |

Na prática, prefira as variantes que retornam **null** (offer, poll, peek) quando fila vazia é um estado normal. Use as variantes que lançam exceção (add, remove, element) quando fila vazia indica um bug — a exceção documenta a expectativa no código.

## 6.3 API de Deque<T> — a superestrutura

A **Deque<T>** duplica todas as operações da Queue com sufixos First e Last. Ela também fornece os métodos **push()** e **pop()** que mapeiam exatamente para a semântica de Pilha — tornando **ArrayDeque** o substituto direto e recomendado da classe legada **Stack**.

```
// ArrayDeque como PILHA (LIFO):
Deque<String> pilha = new ArrayDeque<>();
pilha.push("A");          // addFirst("A")
pilha.push("B");          // addFirst("B")
String topo = pilha.pop(); // removeFirst() → "B"  (LIFO)

// ArrayDeque como FILA (FIFO):
Deque<String> fila = new ArrayDeque<>();
fila.offer("A");          // addLast("A")
fila.offer("B");          // addLast("B")
String frente = fila.poll(); // removeFirst() → "A"  (FIFO)

// ArrayDeque como DEQUE (acesso nas duas extremidades):
Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(10);  deque.addLast(20);  deque.addFirst(5);
// deque: [5, 10, 20]
deque.peekFirst();   // 5  (sem remover)
deque.pollLast();    // 20 (remove)
deque.pollFirst();   // 5  (remove)
```

## 6.4 Comparativo: nossas implementações vs. JCF

| Aspecto | Nossas implementações | JCF (ArrayDeque / LinkedList) |
|---|---|---|
| Contrato | Pilha<T> / Fila<T> (nossas interfaces) | Queue<T> / Deque<T> (java.util) |
| Implementação interna | Object[] com redimensionamento / Node<T> | Object[] com redimensionamento / Node (privado) |
| Complexidade das operações | Θ(1) / Θ(1) amortizado | Θ(1) / Θ(1) amortizado — idêntica |
| Aceita null como elemento | Sim (não verificamos) | ArrayDeque: Não. LinkedList: Sim |
| Iterator / for-each | Não implementado | Sim, com Iterator e ListIterator bidirecional |
| Thread-safety | Não | Não — use LinkedBlockingDeque para concorrência |
| Integração com Stream API | Não | Sim (stream(), parallelStream()) |
| Serialização | Não | Sim (Serializable) |
| Fail-fast (modCount) | Não | Sim — detecta modificações concorrentes |

**Quando usar a implementação própria vs. JCF?**

Em produção com Spring Boot, use sempre o JCF (ArrayDeque para pilha/fila de uso geral). A implementação própria serve para: (a) entender os internals antes de usar a biblioteca; (b) cenários onde o contrato precisa ser restrito — expor apenas push/pop, sem os dezenas de métodos do Deque; (c) contextos educacionais onde a transparência da implementação é o objetivo.

**Regra prática:** declare variáveis com o tipo de interface mais restrito que o código-cliente precisa — **Pilha<T>** ou **Fila<T>** quando estiver usando suas implementações; **Deque<T>** ou **Queue<T>** quando usar o JCF. Isso deixa a intenção explícita e facilita trocas de implementação.

## 7. Testes com JUnit 5

Testar uma estrutura de dados é tão importante quanto implementá-la. Os casos de borda — operação em estrutura vazia, estrutura com um único elemento, redimensionamento — são exatamente onde bugs se escondem. A suite abaixo cobre **PilhaArray<T>** e **FilaCircular<T>**; você deve criar suites equivalentes para **PilhaLista<T>** e **FilaLista<T>** como exercício.

```
package estruturas;
```

```java
import org.junit.jupiter.api.*;
import java.util.EmptyStackException;
import java.util.NoSuchElementException;
import static org.junit.jupiter.api.Assertions.*;

class PilhaArrayTest {

    private PilhaArray<String> pilha;

    @BeforeEach
    void setUp() { pilha = new PilhaArray<>(); }

    // ── Comportamento básico ──────────────────────────────────────────

    @Test @DisplayName("Pilha recém-criada deve estar vazia")
    void deveEstarVaziaAoCriar() {
        assertTrue(pilha.isEmpty());
        assertEquals(0, pilha.size());
    }

    @Test @DisplayName("push e pop devem seguir LIFO")
    void pushEPopDevemSeguirLifo() {
        pilha.push("A");
        pilha.push("B");
        pilha.push("C");
        assertEquals("C", pilha.pop());   // último entrou, primeiro sai
        assertEquals("B", pilha.pop());
        assertEquals("A", pilha.pop());
        assertTrue(pilha.isEmpty());
    }

    @Test @DisplayName("peek não deve remover o elemento")
    void peekNaoDeveRemover() {
        pilha.push("X");
        assertEquals("X", pilha.peek());
        assertEquals(1, pilha.size());    // ainda está lá
        assertEquals("X", pilha.pop());   // agora remove
        assertTrue(pilha.isEmpty());
    }

    // ── Casos de borda ────────────────────────────────────────────────

    @Test @DisplayName("pop em pilha vazia deve lançar EmptyStackException")
    void popEmVaziaDeveriaLancarExcecao() {
        assertThrows(EmptyStackException.class, () -> pilha.pop());
    }
```

```
    @Test @DisplayName("peek em pilha vazia deve lançar EmptyStackException")
    void peekEmVaziaDeveriaLancarExcecao() {
        assertThrows(EmptyStackException.class, () -> pilha.peek());
    }

    @Test @DisplayName("Pilha com um único elemento deve funcionar
corretamente")
    void umUnicoElemento() {
        pilha.push("único");
        assertFalse(pilha.isEmpty());
        assertEquals("único", pilha.peek());
        assertEquals("único", pilha.pop());
        assertTrue(pilha.isEmpty());
    }

    // ── Redimensionamento
──────────────────────────────────────────────────

    @Test @DisplayName("Deve redimensionar além da capacidade inicial")
    void deveRedimensionarAutomaticamente() {
        // PilhaArray começa com capacidade 16; inserir 200 elementos
        for (int i = 0; i < 200; i++) pilha.push("item" + i);
        assertEquals(200, pilha.size());
        assertEquals("item199", pilha.peek());  // topo correto após
redimensionamentos
    }

    @Test @DisplayName("Pop após redimensionamento deve manter LIFO")
    void popAposRedimensionamentoDelesManterlIFO() {
        for (int i = 0; i < 50; i++) pilha.push(String.valueOf(i));
        for (int i = 49; i >= 0; i--) {
            assertEquals(String.valueOf(i), pilha.pop());
        }
    }
}

// ── Suite para FilaCircular
──────────────────────────────────────────────────

class FilaCircularTest {

    private FilaCircular<Integer> fila;

    @BeforeEach
    void setUp() { fila = new FilaCircular<>(); }

    @Test @DisplayName("enqueue e dequeue devem seguir FIFO")
```

```
    void enqueueEDequeueSeguemFifo() {
        fila.enqueue(1); fila.enqueue(2); fila.enqueue(3);
        assertEquals(1, fila.dequeue());
        assertEquals(2, fila.dequeue());
        assertEquals(3, fila.dequeue());
        assertTrue(fila.isEmpty());
    }

    @Test @DisplayName("Fila circular deve funcionar após wrap-around dos
ponteiros")
    void deveManterComportamentoCircular() {
        // Provoca wrap-around: insere 16, remove 14, insere mais 14
        for (int i = 0; i < 16; i++) fila.enqueue(i);
        for (int i = 0; i < 14; i++) fila.dequeue();
        for (int i = 100; i < 114; i++) fila.enqueue(i);
        // Na fila devem restar: 14, 15, 100, 101, ..., 113
        assertEquals(16, fila.size());
        assertEquals(14, fila.peek());  // primeiro que sobrou
    }

    @Test @DisplayName("dequeue em fila vazia deve lançar
NoSuchElementException")
    void dequeueEmVaziaDeveriaLancar() {
        assertThrows(NoSuchElementException.class, () -> fila.dequeue());
    }

    @Test @DisplayName("Deve redimensionar mantendo a ordem FIFO")
    void redimensionamentDeveMantOrdemFifo() {
        for (int i = 0; i < 100; i++) fila.enqueue(i);
        for (int i = 0; i < 100; i++) assertEquals(i, fila.dequeue());
    }
}
```

## 8. Exercícios Práticos

Para cada exercício, escreva a solução em Java e pelo menos dois testes JUnit 5 — um caso normal e um caso de borda.

### Exercício 1 — Inversão de sequência com Pilha

Implemente o método **inverter(String texto)** que inverte a string recebida usando exclusivamente as operações da interface **Pilha<Character>**. Não use **StringBuilder.reverse()** nem arrays auxiliares. Em seguida, responda: qual é a complexidade temporal e espacial da solução?

```
public static String inverter(String texto) {
```

```
    Pilha<Character> pilha = new PilhaArray<>();
    // Passo 1: empilhe cada caractere de texto
    // Passo 2: desempilhe e concatene em uma nova String
    // Implemente aqui
}
// Casos de teste esperados:
// inverter("IFTM")  → "MTFI"
// inverter("")      → ""
// inverter("A")     → "A"
```

## Exercício 2 — Avaliador de expressões pós-fixas

Expressões pós-fixas (notação polonesa reversa) eliminam a necessidade de parênteses: os operandos precedem o operador. Por exemplo, "3 4 +" equivale a 3 + 4 = 7. O algoritmo usa uma única Pilha: ao encontrar um número, empilha; ao encontrar um operador, desempilha dois operandos, aplica o operador e empilha o resultado.

```
public static double avaliarPosFixa(String expressao) {
    Pilha<Double> pilha = new PilhaArray<>();
    for (String token : expressao.split(" ")) {
        switch (token) {
            case "+" -> { double b = pilha.pop(), a = pilha.pop(); pilha.push(a
+ b); }
            case "-" -> { double b = pilha.pop(), a = pilha.pop(); pilha.push(a
- b); }
            case "*" -> { double b = pilha.pop(), a = pilha.pop(); pilha.push(a
* b); }
            case "/" -> {
                double b = pilha.pop(), a = pilha.pop();
                if (b == 0) throw new ArithmeticException("Divisão por zero");
                pilha.push(a / b);
            }
            default  -> pilha.push(Double.parseDouble(token));
        }
    }
    return pilha.pop();
}
// "3 4 +"            → 7.0
// "5 1 2 + 4 * + 3 -" → 14.0   (= 5 + (1+2)*4 - 3)
// "15 7 1 1 + - / 3 * 2 1 1 + + -" → 5.0
```

Estenda o método para suportar o operador % (módulo). Por que % consome dois operandos, enquanto um operador de negação unária (se existisse) consumiria apenas um?

## Exercício 3 — Fila usando duas Pilhas

Questão clássica de entrevistas técnicas (Amazon, Google, Meta): implemente uma Fila usando apenas duas instâncias de **PilhaArray<T>** como estrutura interna. A ideia central é usar uma pilha para receber os elementos (entrada) e outra para servir as remoções (saída). Quando saida está vazia e dequeue é chamado, todos os elementos de entrada são transferidos para saida — invertendo a ordem e restaurando a sequência FIFO.

```
public class FilaDuasPilhas<T> implements Fila<T> {

    private final PilhaArray<T> entrada = new PilhaArray<>();
    private final PilhaArray<T> saida   = new PilhaArray<>();

    @Override
    public void enqueue(T elemento) {
        entrada.push(elemento);  // sempre empilha em entrada
    }

    @Override
    public T dequeue() {
        transferirSeNecessario();
        if (saida.isEmpty()) throw new java.util.NoSuchElementException();
        return saida.pop();
    }

    @Override
    public T peek() {
        transferirSeNecessario();
        if (saida.isEmpty()) throw new java.util.NoSuchElementException();
        return saida.peek();
    }

    @Override public boolean isEmpty() { return entrada.isEmpty() &&
saida.isEmpty(); }
    @Override public int size()        { return entrada.size() + saida.size(); }

    // Transferência: move todos de entrada para saida (inverte a ordem)
    private void transferirSeNecessario() {
        if (saida.isEmpty()) {
            while (!entrada.isEmpty()) {
                saida.push(entrada.pop());
            }
        }
    }
}
```

Analise a complexidade de dequeue() no pior caso e no caso amortizado. Em quais padrões de uso essa implementação se torna ineficiente?

## Exercício 4 — Comparação empírica com System.nanoTime()

Escreva um programa que mede o tempo de 1.000.000 de operações push+pop na **PilhaArray**, na **PilhaLista** e no **ArrayDeque** do JCF. Execute o teste três vezes e calcule a média (para reduzir o efeito da compilação JIT e do GC).

```
long medir(Runnable tarefa) {
    tarefa.run();                          // aquecimento da JVM (JIT)
    long inicio = System.nanoTime();
    tarefa.run();
    return System.nanoTime() - inicio;
}

long tempoPilhaArray = medir(() -> {
    PilhaArray<Integer> p = new PilhaArray<>();
    for (int i = 0; i < 1_000_000; i++) { p.push(i); p.pop(); }
});
System.out.printf("PilhaArray:  %.2f ms%n", tempoPilhaArray / 1e6);
```

O resultado confirma a análise teórica? Qual estrutura é mais rápida e por quê? Pense na diferença de localidade de cache entre array e lista encadeada.

## Exercício 5 — Rate Limiter com janela deslizante

O **RateLimiterService** abaixo usa um **Deque<Long>** para manter os timestamps das últimas requisições de cada usuário. Compreenda o código e responda: (a) Por que **peekFirst()** é chamado antes de **pollFirst()**? (b) Por que **ConcurrentHashMap** em vez de **HashMap**? (c) O que acontece com os dados de usuários que pararam de fazer requisições — o mapa cresce indefinidamente?

```
@Service
public class RateLimiterService {

    private static final int    MAX_REQS  = 10;
    private static final long   JANELA_MS = 60_000L;

    // ConcurrentHashMap: leituras e escritas thread-safe sem bloquear o mapa
inteiro
    private final Map<String, Deque<Long>> historico = new
ConcurrentHashMap<>();

    public boolean permitir(String userId) {
        long agora = System.currentTimeMillis();
        Deque<Long> fila = historico.computeIfAbsent(userId, k -> new
ArrayDeque<>());

        // Remove timestamps fora da janela de 1 minuto
```

```
        // peekFirst() antes de pollFirst(): evita remover sem verificar
        while (!fila.isEmpty() && agora - fila.peekFirst() > JANELA_MS) {
            fila.pollFirst();   // O(1)
        }

        if (fila.size() < MAX_REQS) {
            fila.addLast(agora);   // O(1)
            return true;
        }
        return false;
    }
}
```

## 9. Desafios para Casa

Os desafios a seguir exigem autonomia e criatividade. Cada um deve ser acompanhado de testes JUnit 5 com cobertura mínima de 80% das branches.

### Desafio 1 — Pilha com mínimo em O(1)

Implemente uma pilha que, além das operações padrão, oferece **min()** — que retorna o menor elemento da pilha em Θ(1) — sem percorrer todos os elementos. A solução usa duas pilhas internas: a pilha principal e uma pilha espelho que armazena, em cada posição, o menor elemento visto até aquela profundidade.

```
public class PilhaComMinimo<T extends Comparable<T>> {
    private final PilhaArray<T> principal = new PilhaArray<>();
    private final PilhaArray<T> minimos   = new PilhaArray<>();

    public void push(T elemento) {
        principal.push(elemento);
        // empilha em minimos o menor entre o elemento e o mínimo atual
        if (minimos.isEmpty() || elemento.compareTo(minimos.peek()) <= 0)
            minimos.push(elemento);
        else
            minimos.push(minimos.peek());  // replica o mínimo atual
    }

    public T pop() {
        minimos.pop();
        return principal.pop();
    }

    public T min()  { return minimos.peek(); }
    public T peek() { return principal.peek(); }
```

```
    public boolean isEmpty() { return principal.isEmpty(); }
}
```

Escreva testes que verifiquem min() após cada push e após cada pop, garantindo que ele retorna sempre o mínimo correto. Teste o caso especial em que o mínimo é removido — o novo mínimo deve ser corretamente exibido.

## Desafio 2 — Buffer circular de logs

Implemente **BufferCircular<T>** com capacidade máxima fixa. Quando o buffer está cheio e um novo elemento é adicionado, o mais antigo é descartado automaticamente. Esse padrão é usado em sistemas de monitoramento (guardar as últimas N métricas), em buffers de áudio/vídeo e em janelas deslizantes para rate limiting.

```
public class BufferCircular<T> {
    private final Object[] dados;
    private int inicio, fim, quantidade;

    public BufferCircular(int capacidade) {
        dados = new Object[capacidade];
    }

    public void adicionar(T elemento) {
        dados[fim] = elemento;
        fim = (fim + 1) % dados.length;
        if (quantidade == dados.length)
            inicio = (inicio + 1) % dados.length;  // descarta o mais antigo
        else
            quantidade++;
    }

    // Implemente: removerMaisAntigo(), estaVazio(), estaCheio(), tamanho()
}

// Teste de uso:
BufferCircular<String> logs = new BufferCircular<>(3);
logs.adicionar("INFO: Servidor iniciado");
logs.adicionar("INFO: Conexão aceita");
logs.adicionar("WARN: Timeout na query");
logs.adicionar("ERROR: Falha na autenticação");
// O primeiro log foi descartado — buffer contém os 3 mais recentes
```

## Desafio 3 — API REST com fila de tarefas e Spring Boot

Crie uma API REST com Spring Boot que simula um sistema de processamento assíncrono de tarefas. O sistema deve expor os endpoints abaixo e usar FilaLista<Tarefa> como armazenamento em memória. Adicione um @Scheduled que processa automaticamente uma tarefa a cada 5 segundos, simulando um worker.

```
POST   /api/tarefas           → Adiciona tarefa à fila (retorna 202 Accepted)
GET    /api/tarefas/proxima   → Processa (remove) a próxima tarefa (200 OK)
GET    /api/tarefas/status    → Retorna quantidade de tarefas pendentes
GET    /api/tarefas/historico → Últimas 10 tarefas processadas (use Pilha)
```

Ao finalizar, reflita: quais problemas surgiriam em produção com esse modelo em memória (sem banco de dados ou broker de mensagens)? Como o Spring AMQP com RabbitMQ resolve cada um desses problemas?

### Desafio 4 — Problema de Josephus com lista circular

N pessoas estão em círculo. A partir de uma pessoa inicial, conta-se k posições no sentido horário; a pessoa na posição k é eliminada. O processo se repete até restar uma única pessoa. Use a ListaCircular<T> da Semana 5 para resolver o problema e determine a posição do sobrevivente.

```java
/**
 * Resolve o problema de Josephus com N pessoas e passo k.
 * @return posição (0-indexada) do sobrevivente
 */
public static int josephus(int n, int k) {
    ListaCircular<Integer> circulo = new ListaCircular<>();
    for (int i = 0; i < n; i++) circulo.adicionar(i);

    while (circulo.tamanho() > 1) {
        // Gire k-1 vezes (o próximo a ser eliminado fica na frente)
        for (int i = 0; i < k - 1; i++) circulo.girar();
        circulo.removerInicio();
    }
    return circulo.primeiro();
}
// josephus(7, 3) → 3  (posição clássica do problema original)
// josephus(6, 2) → 4
```

## 10. Armadilhas Clássicas

Esta seção cataloga os erros mais comuns cometidos ao implementar e usar Pilhas e Filas em Java. Cada armadilha é apresentada com o padrão incorreto, a explicação do problema e a correção recomendada.

### Armadilha 1 — Usar java.util.Stack em código novo

**O problema:** A classe **java.util.Stack** é legada desde o Java 1.0 e tem três problemas sérios: (a) estende **Vector**, que é sincronizado por padrão — todas as operações adquirem um lock, causando contenção desnecessária em aplicações single-threaded; (b) herda métodos de **Vector** como **get(int)** e **insertElementAt()**, que violam a semântica de pilha ao permitir acesso por índice; (c) o próprio Javadoc do JDK recomenda **ArrayDeque** como substituto desde o Java 6.

```
// ERRADO — Stack legada: síncrona, expõe get(int), herança incorreta
Stack<String> pilha = new Stack<>();
pilha.get(0);       // viola a semântica de pilha — acesso por índice!

// CORRETO — ArrayDeque como pilha: push/pop/peek idênticos, sem lock
Deque<String> pilha = new ArrayDeque<>();
pilha.push("A");
pilha.pop();
```

### Armadilha 2 — Fila com array sem circularidade (dequeue em O(n))

**O problema:** A implementação ingênua de fila com array usa o índice 0 como início fixo. A cada **dequeue()**, todos os elementos são deslocados uma posição para a esquerda — custo $\Theta(n)$. Para n operações, o custo total é $\Theta(n^2)$. Com 1 milhão de elementos, isso representa bilhões de operações que o array circular eliminaria completamente.

```
// ERRADO — dequeue() em O(n): desloca todos os elementos
public T dequeue() {
    T removido = (T) elementos[0];
    // System.arraycopy move todos os n-1 elementos — Θ(n)
    System.arraycopy(elementos, 1, elementos, 0, quantidade - 1);
    elementos[--quantidade] = null;
    return removido;
}

// CORRETO — dequeue() em Θ(1): apenas avança o ponteiro inicio
public T dequeue() {
    T removido = (T) elementos[inicio];
    elementos[inicio] = null;
    inicio = (inicio + 1) % elementos.length;  // wrap-around circular
    quantidade--;
    return removido;
}
```

### Armadilha 3 — Esquecer de anular referências após pop/dequeue

**O problema:** Ao fazer **pop()** ou **dequeue()**, se o slot do array não for explicitamente atribuído a **null**, o array interno continua guardando a referência ao objeto removido. O Garbage Collector não pode coletar o objeto enquanto o array existir — caracterizando um *memory leak* sutil. Com objetos grandes (imagens, documentos) ou com pilhas de longa duração (uma pilha de sessão HTTP, por exemplo), esse leak pode levar a **OutOfMemoryError** em produção.

```
// ERRADO — mantém referência "morta" após pop
public T pop() {
    return (T) elementos[--topo];   // slot ainda aponta para o objeto!
}

// CORRETO — anula referência para permitir coleta pelo GC
public T pop() {
    topo--;
    T removido = (T) elementos[topo];
    elementos[topo] = null;   // GC pode coletar agora
    return removido;
}
```

### Armadilha 4 — Não realinhar o array ao redimensionar FilaCircular

**O problema:** Ao redimensionar uma fila circular, a simples cópia **Arrays.copyOf(elementos, novoTamanho)** preserva as posições físicas, mas os elementos podem estar fragmentados em duas partes (antes e depois do índice **inicio**). Por exemplo: se **inicio = 10** e o array tem 16 posições, os primeiros 6 elementos lógicos estão nos índices 10–15 e os demais em 0–9. Copiar diretamente resulta em ordem incorreta na fila.

```
// ERRADO — copia posições físicas sem considerar o wrap-around
private void redimensionarERRADO() {
    Object[] novo = Arrays.copyOf(elementos, elementos.length * 2);
    // PROBLEMA: se inicio=10, a fila lógica começa no índice 10,
    // mas o novo array ainda trata 0 como início — ordem incorreta!
    elementos = novo;
}

// CORRETO — realinha os elementos lógicos a partir do índice 0
private void redimensionar() {
    Object[] novo = new Object[elementos.length * 2];
    for (int i = 0; i < quantidade; i++) {
        // (inicio + i) % length mapeia posição lógica → posição física
        novo[i] = elementos[(inicio + i) % elementos.length];
    }
    inicio    = 0;
```

```
    fim       = quantidade;
    elementos = novo;
}
```

**Armadilha 5 — Usar LinkedList quando ArrayDeque é a escolha correta**

**O problema:** Muitos desenvolvedores escrevem **new LinkedList<>()** para criar uma fila ou pilha por hábito, sem perceber que **ArrayDeque** é consistentemente mais rápida para essa finalidade. A **LinkedList** aloca um novo objeto **Node** a cada inserção — pressão constante no Garbage Collector. O **ArrayDeque** usa um array circular e amortiza a alocação; além disso, seus elementos são contíguos na memória, favorecendo o cache do processador.

**A exceção:** use **LinkedList** quando precisar simultaneamente da interface **List<T>** (acesso por índice) e **Deque<T>** na mesma estrutura — situação rara em código bem projetado.

```
// EVITE para uso puro de pilha ou fila:
Deque<String> fila = new LinkedList<>();    // aloca Node a cada offer()

// PREFIRA para uso puro de pilha ou fila:
Deque<String> fila = new ArrayDeque<>();   // array circular, cache-friendly

// Benchmark indicativo (1 milhão de push+pop, JDK 17, JMH):
// ArrayDeque:   ~28 ms
// LinkedList:   ~62 ms   (2,2× mais lento — alocação de Node + GC)
```

## 11. Conexão com Árvores Binárias

Na próxima semana iniciaremos as Estruturas Não Lineares com Árvores Binárias: conceitos, propriedades e percursos (in-order, pre-order, post-order, level-order). As estruturas desta semana serão usadas diretamente na implementação dos dois principais algoritmos de percurso.

### 11.1 Percurso BFS usa Fila (level-order)

O percurso BFS (Breadth-First Search, ou busca em largura) visita os nós de uma árvore nível a nível — primeiro todos os nós do nível 0, depois todos do nível 1, e assim por diante. O algoritmo usa uma **Fila**: começa enfileirando a raiz; a cada iteração, desenfileira um nó, processa-o e enfileira seus filhos. A Fila garante que os nós de cada nível sejam processados antes dos nós do próximo nível, porque os filhos entram no final da fila (FIFO) enquanto os pais são removidos do início.

```
// Esboço do percurso level-order com Fila — veremos em detalhes na Semana 7
public void levelOrder(No<T> raiz) {
    if (raiz == null) return;
```

```
    Fila<No<T>> fila = new FilaLista<>();
    fila.enqueue(raiz);
    while (!fila.isEmpty()) {
        No<T> atual = fila.dequeue();
        System.out.print(atual.dado + " ");
        if (atual.esquerdo != null) fila.enqueue(atual.esquerdo);
        if (atual.direito  != null) fila.enqueue(atual.direito);
    }
}
```

### 11.2 Percurso DFS iterativo usa Pilha

O percurso DFS (Depth-First Search, busca em profundidade) implementado recursivamente usa implicitamente a pilha de chamadas da JVM. Quando convertemos a recursão para uma versão iterativa — para evitar **StackOverflowError** em árvores muito profundas — usamos uma **Pilha** explícita que simula exatamente o comportamento da pilha de chamadas: ao visitar um nó, empilhamos seus filhos; o filho empilhado por último (direito, por convenção) será visitado depois do esquerdo, preservando a ordem de percurso.

```
// Percurso pre-order iterativo — substitui a recursão pela Pilha explícita
public void preOrderIterativo(No<T> raiz) {
    if (raiz == null) return;
    Pilha<No<T>> pilha = new PilhaArray<>();
    pilha.push(raiz);
    while (!pilha.isEmpty()) {
        No<T> atual = pilha.pop();
        System.out.print(atual.dado + " ");
        // Empilha direito primeiro — será processado depois do esquerdo (LIFO)
        if (atual.direito  != null) pilha.push(atual.direito);
        if (atual.esquerdo != null) pilha.push(atual.esquerdo);
    }
}
```

**Tarefa de preparação para a próxima aula**

Resolva os Exercícios 1 a 5 e pelo menos dois Desafios. Certifique-se de que **mvn test** (ou o botão de testes do VS Code) executa todos os casos sem falhas antes de fazer o commit. Na próxima aula, a **FilaLista<T>** e a **PilhaArray<T>** implementadas aqui serão usadas diretamente nos algoritmos de percurso de árvore — trazê-las prontas e testadas é o pré-requisito para a aula.

## 12. Referências Bibliográficas

1. **CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C.** Introdução a Algoritmos: tradução da 4ª edição americana. Rio de Janeiro: Elsevier, 2024. Seções 10.1 (Pilhas) e 10.2 (Filas).
2. **GOODRICH, M. T.; TAMASSIA, R.** Estruturas de Dados e Algoritmos em Java. 6. ed. Porto Alegre: Bookman, 2013. Capítulo 6 (Pilhas, Filas e Deques).
3. **LAFORE, R.** Estruturas de Dados e Algoritmos em Java. 2. ed. Rio de Janeiro: Ciência Moderna, 2004. Capítulos 4 (Pilhas) e 5 (Filas).
4. **BLOCH, J.** Java Efetivo. 3. ed. Rio de Janeiro: Alta Books, 2019. Item 18 (prefira composição à herança — contexto para evitar herdar Stack de Vector). Item 7 (elimine referências obsoletas de objetos).
5. **Oracle.** Java Platform SE 17 API — interface Deque<E>. Disponível em: https://docs.oracle.com/en/java/javase/17/docs/api/. Acesso em: mar. 2026.
6. **Spring Framework.** Spring AMQP Reference Documentation. Disponível em: https://spring.io/projects/spring-amqp. Acesso em: mar. 2026.
7. **OpenJDK.** Código-fonte de ArrayDeque<E>. Disponível em: https://github.com/openjdk/jdk. Acesso em: mar. 2026.

## 13. Glossário

Os termos a seguir aparecem na apostila com sentido técnico específico. Conhecê-los com precisão é necessário tanto para provas e entrevistas técnicas quanto para leitura de documentação em inglês.

| Termo | Definição |
|---|---|
| **LIFO (Last In, First Out)** | Política de acesso em que o último elemento inserido é o primeiro a ser removido. Define o comportamento da Pilha. |
| **FIFO (First In, First Out)** | Política de acesso em que o primeiro elemento inserido é o primeiro a ser removido. Define o comportamento da Fila. |
| **push / pop** | Operações fundamentais da Pilha: push empilha no topo; pop remove e retorna o topo. Ambas em Θ(1). |
| **enqueue / dequeue** | Operações fundamentais da Fila: enqueue insere no final; dequeue remove e retorna o início. Ambas em Θ(1). |
| **peek** | Consulta o elemento no topo (Pilha) ou no início (Fila) sem removê-lo. Θ(1). Também chamado de top() ou front(). |
| **Deque** | Double-Ended Queue (pronúncia "deck"): estrutura que permite inserção e remoção em ambas as extremidades em Θ(1). Superestrutura que engloba Pilha e Fila. |
| **Fila circular** | Implementação de Fila com array em que dois ponteiros (inicio e fim) circulam usando o operador módulo, eliminando a necessidade de deslocamento de elementos. |
| **Complexidade amortizada** | Custo médio de uma operação ao longo de uma sequência longa, quando operações ocasionalmente caras (ex.: redimensionamento em O(n)) são diluídas sobre muitas operações baratas. Resultado: Θ(1) amortizado por operação. |

| Termo | Definição |
|---|---|
| **Type erasure** | Mecanismo do compilador Java que remove informações de tipo genérico (T) em tempo de execução, substituindo T por Object no bytecode. Causa a impossibilidade de criar arrays genéricos (new T[]). |
| **Memory leak (vazamento de memória)** | Condição em que objetos que não são mais necessários continuam sendo referenciados, impedindo o Garbage Collector de recuperar a memória. Em Pilhas/Filas com array, ocorre quando o slot não é anulado após remoção. |
| **Cache locality** | Propriedade de estruturas baseadas em array de armazenar elementos em posições contíguas de memória, favorecendo o carregamento de linhas de cache do processador e acelerando o acesso sequencial em relação a estruturas encadeadas. |
| **FIFO assíncrono (broker de mensagens)** | Implementação distribuída do TAD Fila em ferramentas como RabbitMQ e AWS SQS. Produtores inserem mensagens; consumidores as processam de forma independente, desacoplando o tempo de publicação do tempo de processamento. |
| **Stack trace** | Sequência de frames de método registrada na pilha de execução da JVM no momento de uma exceção. Lida do topo (ponto do erro) até a base (ponto de entrada do thread), espelha o conteúdo da call stack. |